Testkonzept - TicTacToe

Dieses Testkonzept beschreibt die Testziele, Teststrategie, Teststruktur und die vorhandenen automatisierten Testfälle der Java-TicTacToe-Anwendung. Verwendet werden JUnit 5, AssertJ und parametrisierte Tests.

1. Testziele

Ziel ist es, die zentrale Spiellogik zuverlässig zu prüfen. Fehler bei der Gewinnerkennung, ungültige Spielerkonfigurationen und unerwartete Spielabläufe sollen durch automatisierte Tests früh erkannt werden.

Testziel	Beschreibung
Gewinnsituationen erkennen	isWin() soll horizontale, vertikale und diagonale Gewinnsituationen korrekt erkennen.
Falsche Gewinner vermeiden	Board-Konstellationen ohne Gewinner müssen für CROSS und CIRCLE false liefern.
Ungültige Spielerkonfiguration erkennen	Dieselbe Spielerinstanz darf nicht gleichzeitig für CROSS und CIRCLE verwendet werden.
Spielablauf prüfen	Zwei GreedyPlayer sollen einen reproduzierbaren Spielablauf erzeugen; im getesteten Fall gewinnt CROSS.
Mehrere Boards prüfen	Parameterized Tests sollen dieselbe Testlogik mit verschiedenen Board-Konstellationen ausführen.

2. Teststrategie

Es werden automatisierte Unit Tests eingesetzt. Der Schwerpunkt liegt auf TicTacToeMain.isWin() und TicTacToeMain.play(). JUnit 5 führt die Tests aus; AssertJ prüft die erwarteten Resultate.

```
.\gradlew.bat test
```

Testart	Zweck	Beispiel
Positivtest	Prüft korrektes erwartetes Verhalten.	Winning Boards liefern true.
Negativtest	Prüft, dass ein nicht vorhandener Zustand nicht fälschlich erkannt wird.	Non-winning Boards liefern false.
Exception-Test	Prüft die Reaktion auf eine ungültige Verwendung.	Gleiche Spielerinstanz für beide Seiten führt zu IllegalArgumentException.
Parameterized Test	Führt dieselbe Testmethode mit mehreren Testdatensätzen aus.	winningBoards() und nonWinningBoards().

3. Teststruktur

Die Tests folgen dem Given-When-Then-Prinzip. Dadurch sind Vorbereitung, Ausführung und Überprüfung klar getrennt.

Phase	Bedeutung
Given	Board, Spieler und weitere Testdaten werden vorbereitet.
When	Die zu testende Methode, zum Beispiel isWin() oder play(), wird ausgeführt.
Then	Das tatsächliche Ergebnis wird mit AssertJ gegen das erwartete Ergebnis geprüft.

Helpers und Fixtures

Der Helper greedyPlayer() kapselt die Erstellung eines GreedyPlayer. Die Fixtures winningBoards() und nonWinningBoards() liefern vorbereitete Board-Konstellationen für die parametrisierten Tests. Dadurch bleiben die Testmethoden kurz und die Testdaten können zentral erweitert werden.

4. Vorhandene Testfälle

Test	Given	When	Then
isWin_returnsTrueForWinningBoards	Mehrere Boards mit horizontalem, diagonalem oder vertikalem Gewinn.	isWin(board, player)	true
play_rejectsUsingSamePlayerForCrossAndCircle	Dieselbe GreedyPlayer-Instanz wird für beide Seiten verwendet.	play(player, player)	IllegalArgumentException
isWin_returnsFalseForNonWinningBoards	Vollständig belegtes Board ohne Gewinner; Prüfung für CROSS und CIRCLE.	isWin(board, player)	false
play_twoGreedyPlayers_resultsInCrossWinning	Zwei verschiedene GreedyPlayer-Instanzen.	play(crossPlayer, circlePlayer)	Stone.CROSS

5. Parametrisierte Testdaten

winningBoards() enthält aktuell drei Gewinnsituationen: eine horizontale Reihe für CROSS, eine Diagonale für CROSS und eine vertikale Reihe für CIRCLE. nonWinningBoards() verwendet ein Board ohne Gewinner und prüft dieses einmal für CROSS und einmal für CIRCLE. JUnit führt die zugehörige Testmethode für jeden Datensatz separat aus.

6. Erweiterungsmöglichkeiten

Für eine systematischere Abdeckung können weitere Konstellationen ergänzt werden: alle drei horizontalen Reihen, alle drei vertikalen Spalten, beide Diagonalen, ein leeres Board sowie teilweise gefüllte Boards ohne Gewinner. Die vorhandenen parametrisierten Tests ermöglichen diese Erweiterung ohne zusätzliche, fast identische Testmethoden.

7. Erfolgskriterien

Das Testziel gilt als erfüllt, wenn alle definierten Tests automatisiert ausführbar sind, die erwarteten Ergebnisse liefern und der Gradle-Testlauf mit BUILD SUCCESSFUL endet. Wird die geprüfte Spiellogik absichtlich fehlerhaft verändert, soll mindestens der dazugehörige Test fehlschlagen.